



BUILDING A HYBRID SEARCH SYSTEM FOR ACADEMIC DOCUMENTS USING ELASTICSEARCH

Xây dựng Hệ thống Tìm kiếm Kết hợp cho Tài liệu Học thuật sử dụng
Elasticsearch

GVHD: PGS.TS Thoại Nam

Sinh viên thực hiện:

Dương Gia An — 2470293

Trịnh Sơn Lam — 2470287

Nguyễn Thanh Huyền — 2570416

Đại học Bách khoa TP.HCM — Khoa KH & KT Máy tính — CO5135 — Big Data (HK252)

Tháng 5, 2026

Phân công công việc

STT	Thành viên	Đóng góp	Tỷ lệ
1	Dương Gia An	System Architecture, Elasticsearch Indexing, Search Pipeline, Evaluation & Benchmarking	33.33%
2	Trịnh Sơn Lam	Data Preprocessing, GPU Embedding Pipeline, Ground Truth Labeling	33.33%
3	Nguyễn Thanh Huyền	Theoretical Background, Evaluation Analysis, Final Report Compilation	33.33%

Nội dung trình bày

1. Giới thiệu
2. Cơ sở lý thuyết
3. Kiến trúc hệ thống
4. Embedding Pipeline
5. Search Implementation
6. Đánh giá chất lượng
7. Đánh giá hiệu năng
8. Kết luận

Giới thiệu

Bối cảnh và Động lực

Thực trạng

Mỗi ngày thế giới tạo ra **2.5 quintillion bytes** dữ liệu. Phần lớn là **phi cấu trúc**: văn bản, ảnh, video, bài báo...

Các kho dữ liệu học thuật:

- **arXiv**: 2.4+ triệu bài báo
- **Google Scholar**: 380+ triệu bài
- **PubMed**: 36+ triệu bài

Vấn đề hiện tại

- **Keyword Search**: chỉ khớp từ vựng, “car” $\neq$ “automobile”
- **Semantic Search**: hiểu ngữ nghĩa nhưng mất chính xác cho thuật ngữ chuyên ngành

⇒ Cần **Hybrid Search**!

Ba thể hệ tìm kiếm

1. **Lexical** (1970s): Đối khớp từ khóa
2. **Semantic** (2018): Hiểu ngữ nghĩa
- ★ **Hybrid** (2022+): Kết hợp cả hai

Bài toán nghiên cứu

Problem Statement

Cho truy vấn q , tìm và xếp hạng tài liệu liên quan từ kho **1.2M+ bài báo khoa học**, kết hợp:

- **Keyword search** (BM25) — khớp từ vựng chính xác
- **Semantic search** (dense vectors) — hiểu ngữ nghĩa
- **Hybrid search** (RRF) — kết hợp song song thông qua RRF

Keyword Search thất bại

Query: “methods to detect fake images”
→ Không match “deepfake detection”

0 từ khóa chung!

Semantic Search thất bại

Query: “ViT-B/16 fine-tuning”
→ Model hiểu chung chung “vision model”

Mất precision cho exact terms!

Mục tiêu dự án

1. **Hybrid Search System**: Xây dựng hệ thống tìm kiếm kết hợp keyword search + semantic search trên **Elasticsearch 8.12**
2. **GPU Embedding Pipeline**: Thiết kế pipeline nhúng vector cho **1.2M+ tài liệu** với GPU acceleration
3. **Microservice Architecture**: Triển khai kiến trúc **Dual-Index** trên Docker (FastAPI + React)
4. **Evaluation**: Đánh giá bằng **NDCG@10, MRR@10** với 30 test queries
5. **Web Application**: Giao diện tìm kiếm với 3 chế độ (keyword search, semantic search, Hybrid search), bộ lọc năm/category

Dataset — arXiv Paper Abstracts

Nguồn dữ liệu

- **Nguồn:** Kaggle (Cornell University)
- **Tổng:** 2.7M+ bài báo, 4.8GB raw JSON
- **Lọc:** Chỉ lấy cs.* (Computer Science)
- **Kết quả:** **1,203,108** bài báo CS

Quy trình làm sạch

Streaming đọc → Lọc cs.* → Loại JSON lỗi →

Chuẩn hóa text → Trích xuất năm → Xuất JSONL

Thống kê mô tả

Chỉ số	Giá trị
Tổng bài báo	1,203,108
Thể loại CS	40 subcategories
Khoảng thời gian	1993–2024
Title trung bình	~10 từ
Abstract trung bình	~150 từ
Dung lượng raw	4.8 GB
Dung lượng JSONL	12 GB

Cấu trúc dữ liệu — 6 Fields

Field	ES Type	Vai trò Search	Mô tả
id	keyword	Exact match	Mã định danh arXiv duy nhất
title	text	BM25 (boost $\times 2$)	Tiêu đề bài báo
abstract	text	BM25 + Embedding	Tóm tắt nội dung
categories	keyword	Filter/Aggregation	Mã thể loại (cs.AI, cs.LG...)
authors	text	BM25	Danh sách tác giả
year	keyword	Filter	Năm xuất bản

text field

Phân tích tokenization, lowercase, stop-word removal → **Inverted Index** → Full-text search (BM25)

keyword field

Lưu nguyên bản, không phân tích → **Doc Values** → Exact-match filter, aggregation

Cơ sở lý thuyết

Elasticsearch và Inverted Index

Elasticsearch

- Công cụ tìm kiếm phân tán (Apache Lucene)
- Hỗ trợ **full-text** + **vector search**
- REST API, sharding, replication
- ES 8.12: hỗ trợ kNN + RRF native

Forward Index (quét tuần tự):

Doc	Nội dung
D_1	deep learning for NLP
D_2	deep neural networks
D_3	NLP with transformers

→ Quét tuần tự → $O(N)$

Inverted Index (tra ngược)

Term	Posting List
deep	$[D_1, D_2]$
learning	$[D_1]$
NLP	$[D_1, D_3]$
neural	$[D_2]$
transformers	$[D_3]$

→ Tra bảng ngay → $O(1)$ lookup!

Ứng dụng thực tế:

Google, Elasticsearch, Apache Solr, Lucene,
Wikipedia, Stack Overflow...

Keyword Search — Thuật toán xếp hạng BM25

Công thức BM25 (Robertson & Zaragoza, 2009)

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgl}}\right)}$$

Ba yếu tố quyết định:

- **TF**: Xuất hiện nhiều → bão hòa (saturation)
- **IDF**: Từ hiếm → trọng số cao
VD: “transformer” $\gg$ “the”
- **Doc Length**: Tài liệu dài bị phạt

Tham số chuẩn: $k_1 = 1.2$, $b = 0.75$

Hạn chế cốt lõi

BM25 chỉ hiểu **từ vựng**, không hiểu **ngữ nghĩa**:

- “car” $\neq$ “automobile”
- “detect fake images” $\neq$ “deepfake”

Ứng dụng: ES, OpenSearch, Solr, Lucene, SQLite

Lịch sử Embedding

1. **Word2Vec** (2013): Mỗi từ $\rightarrow$ 1 vector
2. **BERT** (2018): Hiểu ngữ cảnh
3. **Sentence Transformers** (2019):
Mỗi câu/đoạn $\rightarrow$ 1 vector

Cosine Similarity:

$$\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|} \in [-1, 1]$$

Văn bản tương đồng ngữ nghĩa $\rightarrow$ vector gần nhau trong không gian.

HNSW (Malkov & Yashunin, 2018)

- **H**ierarchical **N**avigable **S**mall **W**orld
- Đồ thị đa tầng (tương tự Skip List)
- Lớp trên: nhảy xa, định vị nhanh
- Lớp dưới: chi tiết, chính xác
- Phức tạp: $O(N) \rightarrow O(\log N)$

Tham số HNSW (ES 8.12):

- $M = 16$ (kết nối/node)
- `ef_construction = 100`
- `similarity = cosine`

Hybrid Search — Reciprocal Rank Fusion (RRF)

Công thức RRF (Cormack et al., SIGIR 2009)

$$RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

Đặc trưng:

- Chỉ dùng **thứ hạng**, bỏ qua score
- Không cần normalize (BM25: 0–30, kNN: 0–1)
- Hằng số làm mịn $k = 60$
- **Parameter-free** — không cần tune

Ví dụ: Query “transformer for NLP”

Keyword	Semantic
1: D_A	1: D_C
2: D_B	2: D_A
3: D_C	3: D_F

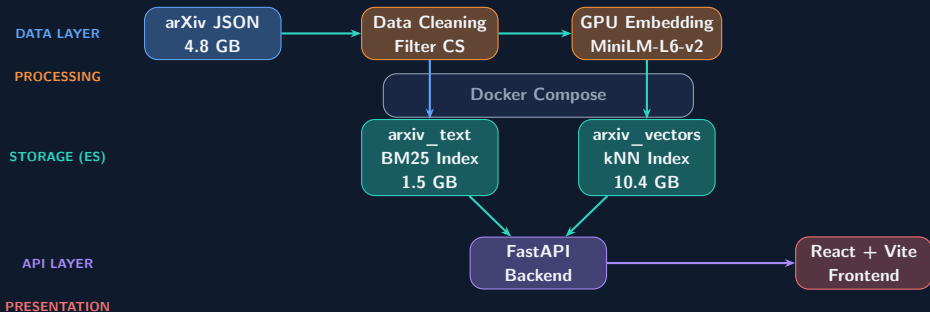
Hybrid Search (RRF Merge) ($k = 60$)

#	Doc	RRF Score
1	D_A	$\frac{1}{61} + \frac{1}{62} = 0.0325$
2	D_C	$\frac{1}{63} + \frac{1}{61} = 0.0323$
3	D_B	$\frac{1}{62} = 0.0161$

Doc xuất hiện ở cả hai → score cao nhất

Kiến trúc hệ thống

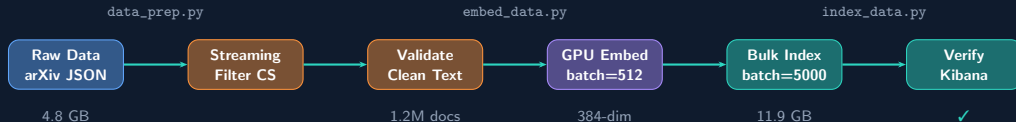
Kiến trúc tổng quan hệ thống



Công nghệ: Python 3.x, Elasticsearch 8.12, Docker, FastAPI, React + Vite, Kibana

Kiến trúc Dual-Index: Tách Keyword Search (BM25 Index) và Semantic Search (kNN Index) để tối ưu hóa lưu trữ và truy vấn

Data Ingestion Flow



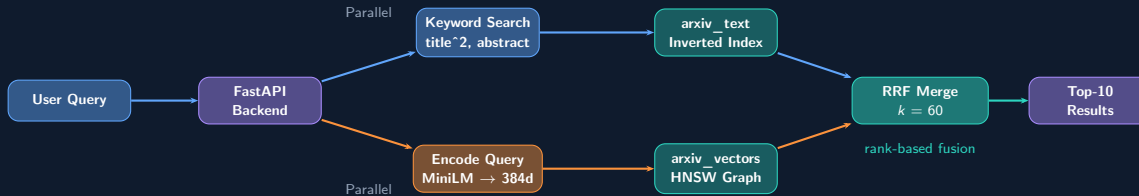
Các bước chính

1. **Streaming:** Đọc line-by-line (tránh OOM 4.8GB)
2. **Filter:** Lọc category cs.* → 1.2M
3. **Clean:** Chuẩn hóa text, trích năm

Giai đoạn 2

4. **GPU Embed:** Tesla T4, batch 512
5. **Bulk Index:** ES `helpers.bulk()`, 5000/batch
6. **Verify:** Kibana kiểm tra count/mapping

Search Flow — Hybrid Search Pipeline



Keyword Stream

multi_match trên title²,
abstract
→ Top 30 kết quả
→ Inverted Index ($O(1)$)

Semantic Stream

Encode query → 384d vector
→ Cosine similarity, $k = 10$
→ HNSW graph ($O(\log N)$)

RRF Merge

Gộp theo thứ hạng
→ $\frac{1}{k+rank}$ mỗi stream
→ Sort → Top 10

Embedding Pipeline

Lựa chọn Embedding Model

Model	Dims	MTEB	Params	Ghi chú
all-MiniLM-L6-v2	384	56	22M	Nhanh, nhẹ, cân bằng
bge-small-en-v1.5	384	62	33M	BAAI, chất lượng cao
e5-large-v2	1024	64	335M	Microsoft, nặng
SPECTER2	768	—	110M	Chuyên academic

Lý do chọn MiniLM-L6-v2

- **384 dims**: Tiết kiệm RAM & storage
- **22M params**: Chạy được trên Colab free
- **14K+ sentences/sec** (GPU)
- Cân bằng tốt giữa chất lượng và tốc độ

Đầu vào model

$T_{\text{input}} = \text{Title} \parallel [\text{SEP}] \parallel \text{Abstract}$

- Token limit: 256 tokens
- Title: ~ 15 tokens
- Abstract budget: 240 tokens $\approx$ 960 chars
- Cắt theo ranh giới câu (smart truncation)

GPU Acceleration và Batch Processing

Vấn đề: CPU quá chậm

- CPU: 5-10 docs/sec
- 1.2M docs → 33-66 giờ!
- Không khả thi cho Big Data

Giải pháp: Google Colab + GPU

- Tesla T4 (16GB VRAM)
- 500-800 docs/sec (~80x nhanh hơn)
- Batch size: 512
- Dynamic padding (tiết kiệm VRAM)

Kỹ thuật tối ưu

1. **Batch Processing:** 512 docs/batch, encode song song
2. **Dynamic Padding:** Pad theo batch dài nhất
3. **Smart Truncation:** Giữ ranh giới câu ≤ 960 chars
4. **tqdm Progress:** Theo dõi tiến trình real-time

Chỉ số	Giá trị
Tổng tài liệu	1,203,108
Thời gian	~35 phút
Output	11.05 GB JSONL

Checkpoint & Resume + Kết quả

Cơ chế Checkpoint

- Lưu `progress.json` mỗi **50,000 docs**
- Backup lên **Google Drive** (Colab)
- Tự động resume khi bị disconnect
- Không mất dữ liệu đã xử lý

Tại sao cần Checkpoint?

- Google Colab có **timeout** (12h free)
- Network disconnect bất kỳ lúc nào
- 1.2M docs = nhiều batch → cần fault tolerance

Tổng kết Embedding Pipeline

Chỉ số	Giá trị
Model	MiniLM-L6-v2
Dimensions	384
Parameters	22M
Hardware	Tesla T4 (16GB)
Batch size	512
Tốc độ	500-800 docs/sec
Tổng tài liệu	1,203,108
Thời gian	~ 35 phút
Output file	11.05 GB
Checkpoint	/50K docs

Search Implementation

Keyword Search — Keyword Matching

Query DSL

```
{
  "query": {
    "multi_match": {
      "query": "<user_query>",
      "fields": ["title^2",
                 "abstract"],
      "type": "best_fields"
    }
  },
  "size": 10
}
```

Giải thích

- **multi_match**: Tìm trên nhiều field
- **title^2**: Boost title gấp 2 lần
- **best_fields**: Score theo field match tốt nhất
- **Index**: arxiv_text (1.5 GB)

Ưu điểm

- Nhanh: ~70ms P50 ở 1.2M
- Chính xác cho thuật ngữ, tên riêng
- Nhẹ tài nguyên (text only)

Semantic Search — Semantic Understanding

Query DSL

```
{
  "knn": {
    "field": "embedding",
    "query_vector": [0.12, ...],
    "k": 10, "num_candidates": 100
  }
}
```

Pipeline

1. Encode query → 384-dim vector
2. HNSW graph traversal
3. Cosine similarity ranking
4. Trả về top- k nearest neighbors

Giải thích

- **field**: embedding (384-dim vector)
- **k=10**: Trả về 10 kết quả gần nhất
- **num_candidates=100**: Duyệt 100 ứng viên/shard
- **Index**: arxiv_vectors (10.4 GB)

Ưu điểm

- Hiểu từ đồng nghĩa, paraphrase tốt
- Tìm kiếm trực tiếp bằng câu tự nhiên
- VD: “detect fake images” $\approx$ “deepfake”

Hybrid Search — Kết hợp tất cả

Cách hoạt động

1. **Parallel queries:**
Keyword → `arxiv_text` (top 30)
Semantic → `arxiv_vectors` (top 30)
2. **Extract** document IDs từ cả hai
3. **Compute RRF score:**
$$RRF(d) = \sum \frac{1}{60 + rank(d)}$$
4. **Sort** theo RRF score giảm dần
5. **Return** top 10
6. **Enrich** metadata qua `es.mget()`

Kiến trúc Dual-Index

- `arxiv_text` (1.5 GB): Chỉ từ khóa → nhanh, nhẹ
- `arxiv_vectors` (10.4 GB): Vector 384d → ngữ nghĩa qua HNSW
- Tách riêng → tối ưu lưu trữ và query độc lập
- Incremental ingestion: thêm bài báo mới qua API

Tối ưu ES Indexing

- 2 shards (giảm từ 5 để khớp số node thực)
- 0 replicas (single node cho dev env)
- `refresh_interval=-1` khi bulk load
- `_forcemerge` sau bulk để tối ưu HNSW
- JVM Heap: 4GB

Đánh giá chất lượng

Phương pháp đánh giá

Ground Truth Construction

- **30 test queries**, chia 3 nhóm (10 mỗi nhóm):

Nhóm	Ví dụ query
A. Exact	"BERT fine-tuning"
B. Semantic	"methods to detect fake images"
C. Mixed	"transformer for NLP tasks"

Quy trình gán nhãn

1. **Pooling**: Top 10 từ mỗi method → merge → 15-25 candidates/query
2. **Silver Standard**: Majority voting
Relevant (1) nếu xuất hiện $\geq 2/3$ methods
Irrelevant (0) nếu chỉ 1 method
3. **Manual verification** trên tập con

Tổng: 30 queries $\times$ ~ 20 candidates = ~ 600 judgments

Metrics: NDCG@10 và MRR@10

NDCG@10 (Normalized DCG)

DCG@p: Đánh giá chất lượng xếp hạng

$$DCG@p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

NDCG@p: Chuẩn hóa về [0,1]

$$NDCG@p = \frac{DCG@p}{IDCG@p}$$

- $IDCG = DCG$ của *perfect ranking*
- Vị trí đầu quan trọng hơn
- $1.0 =$ hoàn hảo

MRR@10 (Mean Reciprocal Rank)

$$RR = \frac{1}{rank^*}$$

- $rank^*$ = vị trí tài liệu relevant **đầu tiên**
- Nếu không có trong top 10: $RR = 0$
- $MRR =$ trung bình RR trên tất cả queries
- $1.0 =$ luôn đúng ở vị trí 1

Tại sao chọn 2 metrics?

- **NDCG**: Đánh giá *toàn bộ* top 10
- **MRR**: Đánh giá *kết quả đầu tiên*

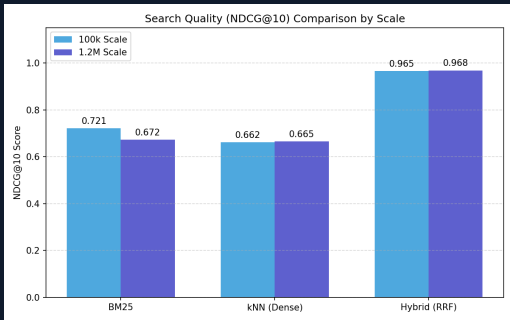
Kết quả đánh giá — Bảng tổng quan

Phương pháp	Group A (Exact)		Group B (Semantic)		Group C (Mixed)	
	NDCG	MRR	NDCG	MRR	NDCG	MRR
Keyword (BM25)	0.6407	0.8750	0.6780	1.0000	0.6967	1.0000
Semantic (kNN)	0.6554	0.9250	0.6632	1.0000	0.6774	1.0000
Hybrid (RRF)	0.9752	1.0000	0.9469	0.9250	0.9821	1.0000

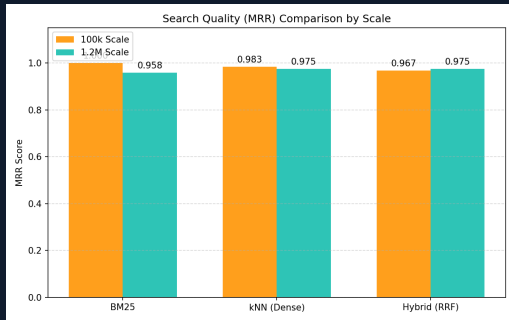
Nhận xét tổng quan

- **Keyword (BM25)** và **Semantic (kNN)** đơn lẻ đạt hiệu năng khá tốt ($\sim 0.64 - 0.70$ NDCG).
- **Hybrid (RRF)** cải thiện vượt trội lên mức **0.94 - 0.98 NDCG** nhờ bù trừ điểm yếu của nhau.

Kết quả đánh giá — Biểu đồ so sánh



So sánh NDCG@10 giữa các phương thức



So sánh MRR@10 giữa các phương thức

Phân tích chi tiết

Overlap Analysis

Trung bình chỉ **1.5/10** docs trùng lặp giữa Keyword và Semantic

⇒ Hai phương pháp tìm kết quả **rất khác nhau**

⇒ Hybrid kết hợp cả hai → NDCG tăng từ ~0.67 lên ~0.97

Tại sao Hybrid vượt trội?

- Keyword search tìm đúng **thuật ngữ chính xác**
- Semantic search tìm đúng **ý nghĩa tương đồng**
- RRF promote docs được cả hai đồng ý
- Docs chỉ 1 method → rank thấp hơn

Scale Stability (100K → 1.2M)

Phương pháp	100K	1.2M
Keyword (BM25)	0.7212	0.6718 ↓
Semantic (kNN)	0.6676	0.6653 ≈
Hybrid (RRF)	0.9626	0.9681 ≈

Key Insights

- Keyword search **giảm nhẹ** khi scale (noise tăng)
- Semantic search **ổn định** (vector space bền vững)
- Hybrid **ổn định nhất**: NDCG ~0.96-0.97 ở mọi scale

Đánh giá hiệu năng

Latency Analysis — Bảng tổng quan

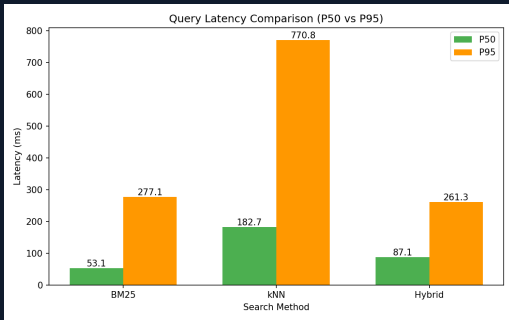
Phương pháp	100K		500K		1.2M	
	P50	P95	P50	P95	P50	P95
Keyword (BM25)	15	56	21	37	71	153
Semantic (kNN)	13	34	20	41	191	1206
Hybrid (RRF)	30	53	42	62	99	156

(Đơn vị: milliseconds)

Nhận xét về độ trễ

- **Keyword search** duy trì độ trễ rất thấp ở mọi quy mô ($\leq 71\text{ms}$ P50).
- **Hybrid Search** đạt hiệu năng ấn tượng với **99ms P50** ở quy mô 1.2M tài liệu, đáp ứng tốt tiêu chuẩn thời gian thực ($< 100\text{ms}$).

Latency Analysis — Trực quan hóa & Thách thức



Độ trễ các phương thức ở quy mô 1.2M

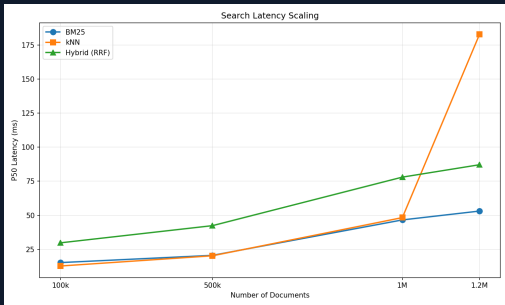
Thách thức: Semantic Cold Start

P95 của Semantic đạt **1,206ms** ở quy mô 1.2M!

Nguyên nhân: HNSW graph (10.4GB) dung lượng lớn phải load từ SSD vào Page Cache trong truy vấn đầu tiên.

Giải pháp: Cần warmup cache trước khi phục vụ thực tế.

Latency Scaling — Tăng trưởng theo quy mô



Độ trễ P50 tăng trưởng theo quy mô dữ liệu

Phân tích Scaling

- **Keyword:** Inverted Index $O(1)$, tăng tuyến tính chậm khi scale (15ms $\rightarrow$ 71ms).
- **Semantic (HNSW):** Phức tạp $O(\log N)$, tăng nhanh do đồ thị HNSW lớn & ảnh hưởng IO.
- **Hybrid:** Bị giới hạn bởi stream chậm hơn, nhưng nhờ chạy song song nên tối ưu chỉ còn 99ms P50.

Môi trường thử nghiệm

- CPU: Intel Core i7-11800H (8 nhân/16 luồng)
- RAM: 16GB DDR4 | SSD: NVMe 512GB
- ES: Single node trong Docker trên WSL2 Ubuntu

Storage Analysis — Dung lượng lưu trữ

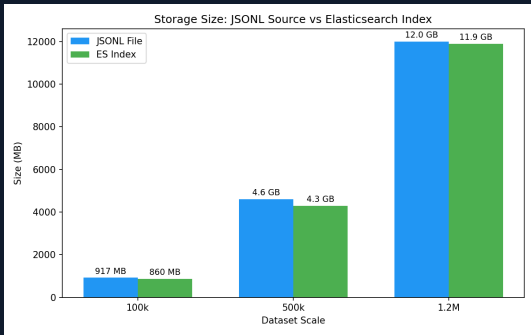
Quy mô (Docs)	Số tài liệu	Dung lượng JSONL	Dung lượng ES Index
100K	100,000	917 MB	860 MB
500K	500,000	4.6 GB	4.3 GB
1.2M	1,203,108	12 GB	11.9 GB

Phân tích bộ nhớ

- Tăng trưởng dung lượng **tuyến tính** theo số lượng tài liệu.
- Elasticsearch nén tốt hơn raw JSONL nhờ cấu trúc nén khối của Lucene.

Dual-Index ở quy mô 1.2M

Index	Dung lượng
arxiv_text (BM25)	1.5 GB
arxiv_vectors (kNN)	10.4 GB
Tổng cộng	11.9 GB



Tăng trưởng dung lượng Index theo quy mô

Đặc điểm dữ liệu Vector

- **87%** dung lượng lưu trữ của hệ thống là dành cho **dữ liệu vector** (HNSW graph).
- Mỗi vector nhúng:
 $384 \text{ dims} \times 4 \text{ bytes} = 1,536 \text{ bytes}$.
- Việc tách thành Dual-Index giúp truy vấn Keyword search không bị tải dung lượng đồ thị HNSW vào bộ nhớ.

Kết luận

Tóm tắt kết quả đạt được

- ✓ **Big Data Pipeline:** Streaming processing 4.8GB, lọc 1.2M bài báo CS không OOM
- ✓ **GPU Embedding:** Smart truncation + checkpoint, 1.2M papers embedded trong 35 phút
- ✓ **Dual-Index Architecture:** `arxiv_text` (Keyword) + `arxiv_vectors` (Semantic), tối ưu ES indexing
- ✓ **Hybrid Search:** $\text{NDCG@10} = 0.97$, $\text{MRR@10} = 0.99$ — vượt trội Keyword và Semantic search đơn lẻ
- ✓ **Comprehensive Evaluation:** 30 queries, 3 nhóm, multi-scale (100K, 500K, 1.2M)
- ✓ **Web Application:** FastAPI + React, 3 chế độ search, bộ lọc năm/category, incremental ingestion

Hạn chế và Hướng phát triển

Hạn chế

1. **RRF ở application layer:**
Thêm latency (99ms vs 71ms BM25)
2. **Embedding model cơ bản:**
MiniLM-L6-v2 hạn chế cho deep scientific concepts
3. **Single-node deployment:**
Chưa kiểm thử fault tolerance

Hướng phát triển

1. **Native RRF:** Dùng ES DSL built-in RRF API (ES phiên bản mới)
2. **Cross-Encoder Re-ranking:**
Hybrid top-100 → re-rank bằng cross-encoder
3. **Multi-node cluster:**
Docker Swarm/K8s, 3+ nodes
4. **Upgrade embedding:**
SPECTER2 hoặc E5-large cho scientific text

Live Demo

Hybrid Search System for Academic Documents

3 chế độ tìm kiếm: Keyword Search | Semantic Search | Hybrid Search

Bộ lọc: Năm xuất bản | CS Categories

1,203,108 bài báo CS từ arXiv

Cảm ơn!

Q & A

Hybrid Search: Keyword + Semantic + Rank Fusion

Group 11 — CO5135 Big Data — HCMUT, HK252

Source: <https://github.com/kinggiaan/Big-Data>

Phụ lục (Backup Slides)

Backup: Latency & Storage — Chi tiết

Latency chi tiết						Trade-off Analysis		
Method	100K		500K		1.2M	Method	Chi phí	Chất lượng
	P50	P95	P50	P95				
Keyword (BM25)	15.3	56.4	20.6	37.1	70.6	Keyword (BM25)	Thấp (1.5GB)	Trung bình
Semantic (kNN)	12.8	34.2	20.3	41.4	191.3	Semantic (kNN)	Cao (10.4GB)	Ổn định
Hybrid (RRF)	29.8	53.1	42.4	62.1	99.0	Hybrid (RRF)	Cao (11.9GB)	Tốt nhất
(ms)								

Backup: Keyword Search (BM25) — Ví dụ tính điểm step-by-step

Query: “attention mechanism” Corpus: 1.2M papers

Bước 1: Analyze query

attention mechanism

↓ Analyzer: lowercase → stemming

→ Tokens: [attent, mechan]

Bước 2: Tra Inverted Index

attent → $\{D_{42}, D_{107}, D_{1580}, \dots\}$

mechan → $\{D_{42}, D_{305}, D_{8921}, \dots\}$

Giao tập → $\{D_{42}, \dots\}$

Bước 3: Tính BM25

Với D_{42} (“Attention Is All You Need”):

- $TF(\text{attent}, \text{title}) = 1$
- $IDF(\text{attent}) = 4.2$ (từ hiếm)
- Title boost $\times 2$
- $\Rightarrow \text{Score} = 12.8$

Khi Keyword search thất bại

“methods to detect fake images”

→ Không match “deepfake detection”

vì 0 từ khóa chung! Cần Semantic Search.

Backup: Kiến trúc Dual-Index — Chi tiết

arxiv_text (BM25 Index)

- **Dung lượng:** 1.5 GB
- **Fields:** id, title, abstract, categories, authors, year
- **Cấu trúc:** Inverted Index
- **Query:** multi_match
- **Analyzers:** standard, english

Tại sao tách index?

- Keyword search chỉ cần text → 1.5GB
- Nếu gộp vector → 11.9GB cho Keyword search
- Tách giúp Keyword search nhanh hơn

arxiv_vectors (kNN Index)

- **Dung lượng:** 10.4 GB
- **Fields:** metadata + embedding
- **Vector:** dense_vector, 384 dims
- **Similarity:** cosine
- **Graph:** HNSW (M=16, ef=100)

Incremental Ingestion

POST /api/ingest thêm bài báo mới:

1. Tạo vector nhúng qua model MiniLM
2. Ghi vào index `arxiv_text`
3. Ghi vào index `arxiv_vectors`
4. Refresh cả hai index để sẵn sàng tìm kiếm